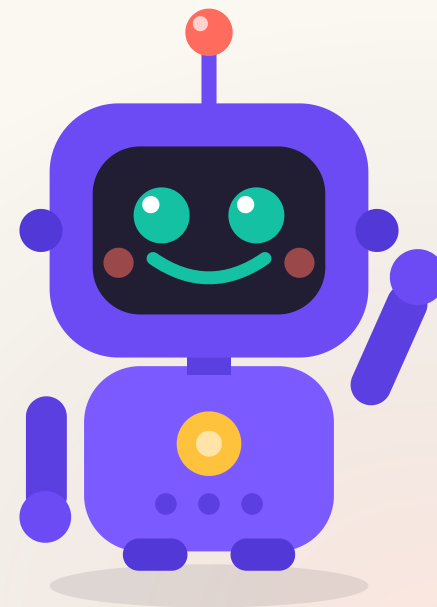


Игра «Угадай число»

Компьютер загадал
число, а ты
угадываешь! Соберём
из циклов настоящую
игру.



Я загадал число! А ты угадай 🎲



Я задумал число от 0 до 99 и никому не скажу. Твоя задача — угадать за поменьше попыток. А я буду подсказывать: **больше** или **меньше**.

Сегодня соберём игру из `while`, `if` и `cin` — всё уже знакомое!

**Компьютер загадал
число от 0 до 99.
Сколько попыток
тебе нужно?**

К концу урока у тебя будет настоящая работающая игра.

Вспомним прошлый урок

- `while` и `for` повторяют тело по условию
- `if (условие) { ... }` — программа принимает решение
- `cin >> x;` — спрашиваем число у игрока

✨ А ЗНАЕШЬ ЧТО?

Соединим всё это в игру: повторяем попытки в цикле, пока игрок не угадает.



Что мы сегодня сделаем

- 1 Научимся загадывать случайное число
- 2 Освоим `break` — выйти из цикла досрочно
- 3 Освоим `continue` — пропустить шаг цикла
- 4 Соберём игру «Угадай число» по частям
- 5 **Практика** — улучшаем игру
- 6 **Домашнее задание**

Случайное число

main.cpp

```
#include <cstdlib>
```

```
int secret = rand() % 100; // число от 0 до 99
```

`rand()` выдаёт случайное число, а `% 100` оставляет остаток от 0 до 99.

СОВЕТ

Знак `%` — это деление с остатком из второго модуля. Остаток деления на 100 всегда меньше 100.

Чтобы числа были разными

```
main.cpp

#include <cstdlib>
#include <ctime>

srand(time(0));           // «перемешать» один раз в начале
int secret = rand() % 100;
```

`srand(time(0))` пишут один раз в самом начале `main` — он заводит генератор по часам.



Без `srand` я каждый запуск загадываю одно и то же число — скучно! С ним — каждый раз новое.

break · ВЫЙТИ ИЗ ЦИКЛА

break досрочно прерывает цикл — как стоп-кран.

```
main.cpp

while (true) {
    std::cin >> guess;
    if (guess == secret) break; // Выходим
}
```

СОВЕТ

Как только число угадано, **break** мгновенно выводит нас из цикла — дальше тело не выполняется.

continue · пропустить шаг

`continue` пропускает остаток шага и переходит к следующему повтору.



main.cpp

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) continue;    // тройку пропускаем  
    std::cout << i << " ";  
}
```

✨ А ЗНАЕШЬ ЧТО?

Выведет  1 2 4 5 — на шаге `i` равно 3 тело пропустилось, а цикл пошёл дальше.

break и continue · в чём разница

break 

полностью **выходит** из цикла

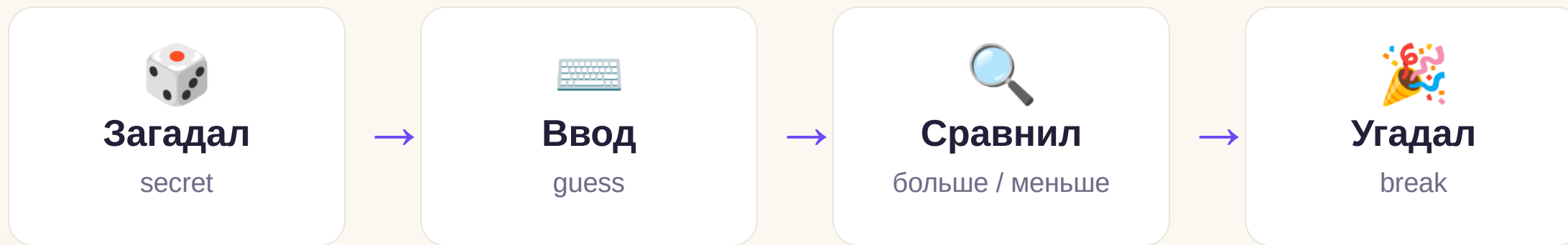
continue 

пропускает один шаг, цикл продолжается



Запомни так: break — стоп всему циклу. continue — «этот шаг пропустим, а дальше работаем».

Собираем игру - логика на пальцах



Повторяем «ввод → сравнение» в цикле, пока игрок не угадает.

Игра целиком · часть 1/2

Сначала загадываем число и готовим переменные:

```
main.cpp

#include <iostream>
#include <cstdlib>
#include <ctime>
```

СОВЕТ

Три **#include** обязательны: `iostream` — для ввода-вывода, `cstdlib` — для `rand`, `ctime` — для `time`.

Игра целиком · часть 2/2

Теперь сам цикл угадывания:

```
main.cpp

while (true) {
    std::cin >> guess;
    if (guess < secret)
        std::cout << "Больше!\n";
    else if (guess > secret)
```



Это вся игра! Соедини часть 1 и часть 2 — и можно играть.

Разбираем главное



main.cpp

```
while (true) {           // цикл «пока не угадаем»
    std::cin >> guess;    // игрок вводит число
    if (...) ...         // подсказка больше/меньше
    else { ...; break; }  // угадал — выходим
}
```



while (true) крутится вечно — выйти из него можно только через **break**. Поэтому **break** тут обязателен, иначе игра не закончится.

Частые ошибки ⚠



Забыл break

`while (true)` без `break` —
игра не кончится.



Перепутал знаки

`<` и `>` местами —
подсказки «врут».



Забыл srand

Без `srand(time(0))` число
всегда одинаковое.

💡 СОВЕТ

Главное в `while (true)` — не забыть `break`, иначе получишь бесконечный цикл.

Проверь себя · что выведет программа? 🧠

```
main.cpp

for (int i = 1; i <= 5; i++) {
    if (i == 2) continue;
    std::cout << i << " ";
}
```

Подумай 10 секунд...

✨ А ЗНАЕШЬ ЧТО?

Выведет ▶ 1 3 4 5 . На шаге i равно 2 сработал continue — двойку пропустили.

Проверь себя · break 🧠

main.cpp

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) break;  
    std::cout << i << " ";  
}
```

А здесь что появится?

✨ А ЗНАЕШЬ ЧТО?

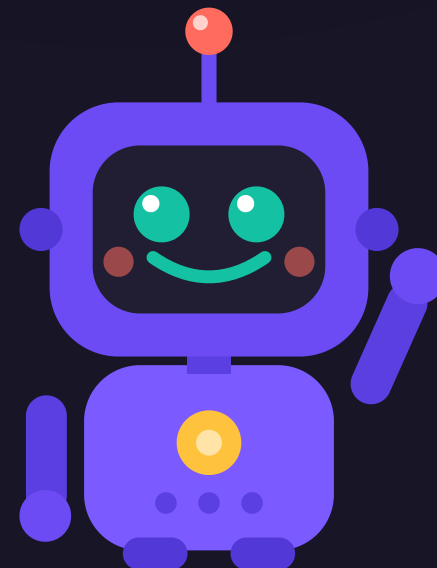
Выведет **▶ 1 2**. На шаге *i* равно 3 сработал `break` — цикл сразу остановился.

ПРАКТИКА В КЛАССЕ

Собираем «Угадай число»



Пишем настоящую игру и улучшаем её своими идеями.



✦ ЗАДАЧА

Задача 1 · Запускаем игру

Собери игру из части 1 и части 2 целиком и сыграй с соседом:



main.cpp

```
srand(time(0));  
int secret = rand() % 100, guess;  
while (true) {  
    std::cin >> guess;
```

Не забудь три `#include` и `main`!

✦ ЗАДАЧА

Задача 2 · Считаем попытки

Добавь в игру счётчик попыток. Заведи `int tries = 0;` до цикла и `tries++` на каждом вводе.

- Когда игрок угадал — выведи: «Готово за ... попыток!»
- Печатай `tries` в финальном сообщении

Подсказка: `tries++` ставим сразу после `cin`

✦ ЗАДАЧА

Задача 3 · Лёгкий уровень

Сделай версию полегче: компьютер загадывает число от 0 до 9 (`rand() % 10`).

- Поменяй только `% 100` на `% 10`
- Сыграй — угадывать стало проще?

Время: 6 минут

✦ ЗАДАЧА

Задача 4 · Ограничь попытки

★ со звёздочкой

Дай игроку только **5 попыток**. Используй `for (int i = 1; i <= 5; i++)` вместо `while (true)`.

- Угадал — выводим «Победа!» и `break`
- Попытки кончились — «Игра окончена, число было ...»

Кто добавит подсказку «осталось N попыток» — мастер!

Что мы сегодня узнали 🦾



Ты собрал настоящую игру из того, что мы уже проходили! Циклы, условия и ввод — и готово.

- `rand() % 100` — случайное число от 0 до 99
- `srand(time(0))` — чтобы числа были разными (один раз в начале)
- `break` — досрочно выйти из цикла
- `continue` — пропустить один шаг цикла

Домашнее задание

1. Реши рабочий лист 4.3 — `break` / `continue` и своя версия игры.
2. Загляни в шпаргалку 4.3 — случайные числа и управление циклом.

Дальше: модуль 5 — функции и массивы: научимся хранить целые списки данных.